Journal of Intelligent Manufacturing (2026) 37:1179–1195
https://doi.org/10.1007/s10845-025-02590-9

A method to benchmark high-dimensional process drift detection

Edgar Wolf[1] · **Tobias Windisch**[1]

Received: 9 August 2024 / Accepted: 3 March 2025 / Published online: 21 April 2025
© The Author(s) 2025

Abstract
Process curves are multivariate finite time series data coming from manufacturing processes. This paper studies machine learning that detect drifts in process curve datasets. A theoretic framework to synthetically generate process curves in a controlled way is introduced in order to benchmark machine learning algorithms for process drift detection. An evaluation score, called the temporal area under the curve, is introduced, which allows to quantify how well machine learning models unveil curves belonging to drift segments. Finally, a benchmark study comparing popular machine learning approaches on synthetic data generated with the introduced framework is presented that shows that existing algorithms often struggle with datasets containing multiple drift segments.

Keywords Machine learning · Drift detection · Process curves · Data synthetization · Manufacturing data · Data modelling · Model evaluation

Introduction

Manufacturing lines typically consist of *processes* arranged sequentially, each using techniques like casting, forming, or joining to shape components to their final specifications. Advanced sensor technology enables precise monitoring of key performance indicators, like force, pressure, or temperature, over time. IoT-enabled systems now commonly store the data obtained, called *process curves*, facilitating analysis across both single components and entire production sequences (Zhong et al., 2017). Issues like anomalous batches, tool wear, or miscalibrations can degrade performance, often subtly, by causing gradual shifts in process curves. Thus, detecting *process drifts* is key to keep unplanned downtimes and scrap parts at bay. In high-volume production, this is particularly challenging due to the rapid data generation and complexity of multi-variable curves and hence these settings have been an ideal application for machine learning methods (Al Assadi et al., 2022; Hesser et al., 2019; Lee et al., 2021; Mayr et al., 2019; Meiners et al., 2021; Yadav et al., 2020; Yong et al., 2020). Although process curves are multivariate time-series, process drift detection should not be confused with drift detection in time series (Lukats et al., 2024) or drifts in profile data (Paynabar & Jin, 2021) (see also Fig. 1). Typically, statistical drift detection methods from time series analysis are not direct applicable, not alone because process curves are high-dimensional objects, but also because of high autocorrelation among their sample axis. As a consequence, deep learning techniques, most prominently dimensionality reduction methods like *autoencoders* (Hyvarinen et al., 2019; Khemakhem et al., 2020; Kim et al., 2023; Kingma & Welling, 2014; Rumelhart et al., 1986), have become increasingly popular (Meiners et al., 2021) as they allow to first learn a low-dimensional representation of the high-dimensional input and then analyse the learned latent variables with classic statistical tools, like sliding Kolmogorov-Smirnov tests (Raab et al., 2020), Hellinger-distance based techniques (Ditzler & Polikar, 2011), or by facilitating the Maximum Mean Discrepancy (Gretton et al., 2012).

For many machine learning applications relevant for manufacturing, established ways and datasets to benchmark the performance of algorithms exist, like for *causal discovery* in quality data (Göbler et al., 2024), *anomaly detection* in images from optical inspections (Bergmann et al., 2021), or *reinforcement learning* in continuous control tasks (Duan et al., 2016). However, for process drift detection, such a framework is yet missing to the best of our knowledge. This may be

✉ Edgar Wolf
edgar.wolf@hs-kempten.de

Tobias Windisch
tobias.windisch@hs-kempten.de

[1] University of Applied Sciences Kempten, Kempten, Germany

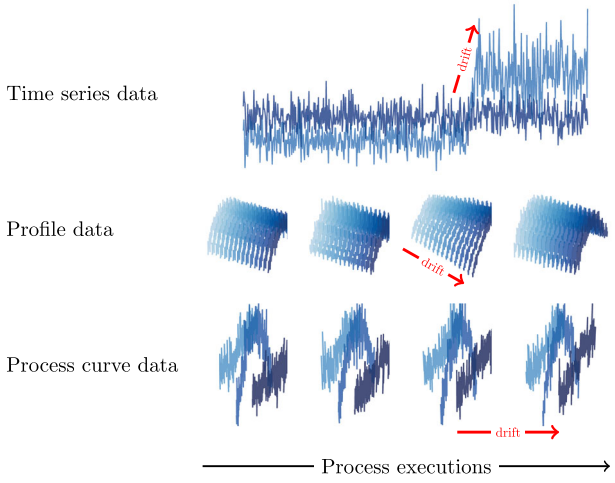


Fig. 1 Overview of different kinds of time series data from manufacturing processes and drifts within

due to the following two reasons: The lack of both, publicly available datasets and a suitable evaluation metric. Beside a few publicly released datasets (Agogino et al., 2007; Mauthe et al., 2024; Tnani et al., 2022), most of the existing work does not release any data to the public, making it impossible to test other detectors on the same dataset. Often, this is due to privacy issues and fear of leaking information to competitors. In addition, as datasets for process drifts are inherently non identically and identically distributed (iid), any sort of test and train splits introduced significant biases making evaluation of algorithms hard if only one variant of a dataset is available. Finally, process drift detection is by definition an unsupervised learning task, but to benchmark detectors, a ground truth is required, labeling precisely when a drift starts and when it ends. This seamlessly leads to the second challenge, namely the missing evaluation metric. As in any machine learning task, the metric depends on the precise application and a trustworthy ground truth. A commonly used metric used in research and practice to measure the statistical performance of a binary classifier, often independent of the application, is the *area under the ROC curve* (Hanley & McNeil, 1982)—short AUC. However, the usage of the AUC is typically only applicable in settings where data is assumed to be iid, unlike in drift detection. In our work, we want to exactly address these issues by introducing a benchmarking framework for researchers, allowing them to reliably validate their process drift detection algorithms. At a high level, our main contributions are:

- We present a simple, yet flexible and effective theoretic framework to generate synthetic process curve datasets including drifts with a validated ground truth (Sects. Statistical framework to model process drifts and Data generation) which also allows feeding of curves from real processes.
- We introduce an evaluation metric called *temporal area under the curve* (TAUC) in Sect. The temporal area under the curve, which aims to take the temporal context of a detection into account.
- We conduct a short benchmark study in Sect. Experiments as a proof of concept for the effectiveness of both, our TAUC metric and our proposed data generation method to measure the predictive power of drift detectors.

Our work is based on preliminary results of the first author (Wolf, 2024). In this work, we provide additionally insights into the introduced metric, introduce a variant called *soft TAUC* and compare it in depth with existing metrics. Moreover, we substantially generalize the data synthetization framework, for instance by allowing higher-order derivatives, and we generate more sophisticated datasets for the benchmark study. We also release the code that helps to generate process curves to benchmark drift detectors, which is freely available under https://github.com/edgarWolf/driftbench. Its optimization back-end is implemented in `Jax` (Bradbury et al., 2018) allowing a fast GPU-based generation of process curves.

Statistical framework to model process drifts

In this section, we formalize what we consider as process curves and drifts within. Generally speaking, process curve datasets are datasets consisting of finitely many multivariate time series each having finitely many steps.

We formally model a process curve as a finite time-series $(Y(x))_{x\in I}$ with $Y(x) \in \mathbb{R}^c$, $I \subset \mathbb{R}$ a finite set, and where $Y : \mathbb{R} \to \mathbb{R}^c$ represent physical properties of the process to be measured and x an independent variable, often the time. In staking processes, for instance, Y is the measured force and x the walked path of the press (compare also Göbler et al., 2024, Fig. 9). Another example are pneumatic test stations, where Y might be a pressure measured over time x. In bolt fastening processes, Y represents the torque measured over the angle x (Meiners et al., 2021). We call the number of variables $c \in \mathbb{N}$ in the curve the *dimension* of the process curve and write $[T]$ for the set $\{1, \ldots, T\}$ and often refer to it as *temporal axis*.

Whenever a manufacturing process finishes its work on a component, a process curve is yielded. Thus, when the same process is executed on multiple times sequentially, a long sequence $C_1, C_2, \ldots, C_t, \ldots, C_T$ with $T \in \mathbb{N}$ of process curves is obtained where each C_t arises under slightly different physical conditions $Y_1, \ldots, Y_T$, i.e., $C_t = (Y_t(x + \epsilon_x) + \epsilon_y)_{x\in I_t}$, where ϵ_x and ϵ_y represents measurement noise or inaccuracies. In theory, also the sets I_t can vary for each $t \in [T]$, for instance due to different offsets. Wearout or tool degradation affects the process curves gradually and to model

their deformation along the execution axis, we assume that there exists functions $f : \mathbb{R}^k \times \mathbb{R} \to \mathbb{R}^c$ and $w : [T] \to \mathbb{R}^k$ such that for all $t \in [T]$ and $x \in I_t$:

$$f(w(t), x) = Y_t(x) \tag{2.1}$$

where the function f is a proxy for the physics underneath the process. The vector $w(t) \in \mathbb{R}^k$ represents environmental properties of the t-th execution, and some of its coordinates correspond to component properties, some to properties of the machine. Without restricting generality and to keep notation simple, we will assume for the remainder that $c = 1$, as the multivariate case is a straight-forward application of our approach by modeling each variable in Y individually (see also Remark 3.1).

Assuming only component variance and no tool degradation, we could assume that $w(t)$ is sampled in each process execution from a fixed but unknown distribution on $\mathbb{R}^k$, like $w(t) \sim \mathcal{N}_{\mu,\sigma}$ with fixed $\mu \in \mathbb{R}^k$ and $\sigma \in \mathbb{R}^{k\times k}$ for all $t \in [T]$. As mentioned, tool degradation, in contrast, affects the process from execution to execution, i.e., the parameters of the distribution shift over time leading to a deformation of the observed process curve. Such process drifts should not be confused with concept drifts, where the goal is typically to analyse the declining performance of a trained machine learning model when new data starts to differ from the train data (Nunes & Guedes, 2024). Moreover, detecting drifts in process curves is different to detecting drift in *profile data* (Paynabar & Jin, 2021), where one typically is interested in drifts among the curves yielded by a single execution, not in drifts over multiple executions. A similar application is the identification of drifts within profile data, where typically one process execution yields a sequence of process curves of fixed size, like in spectroscopy when one curve is some intensity over time which is measured for different wavelengths (Yue et al., 2018). One way to model process drifts is to model the evolution of the latent parameters $w(t)$, like using a dynamical system. For instance, in control theory (Brogan, 1991), $w(t)$ is considered as latent state of a system which evolves over the executions t and one observes a multivariate output $Y(t) \in \mathbb{R}^{|I_t|}$ with $Y(t) = Y_t(I_t)$. Introducing a control vector $u(t) \in \mathbb{R}^p$, $w(t)$ can be considered as state variable $w(t)$ of the system that evolves over time and is influenced by a control vector $u(t) \in \mathbb{R}^p$ such that $\partial_t w(t) = h(w(t), u(t), t)$ and $Y(t) = f(w(t))$ holds for all $t \in \mathbb{N}$. Here, however, one has to precisely model how the state w changes over executions and how it is affected by interventions u and has to solve challenging non-linear differential equations. However, as we will argue, the degradation of the curve can be described directly in curve space in many scenarios. Thus, we directly model the transformation of the process curves in curve space by letting certain *support points* of the curve move in a controlled way:

Definition 2.1 (*Support points*) Let $f : \mathbb{R}^k \times \mathbb{R} \to \mathbb{R}$ be an i-times differentiable function, $i \in \mathbb{N}$, and $\partial_x^i f$ be the i-th derivative of f according to the second argument. Let $x, y \in \mathbb{R}^n$, then (x, y) is a *support point of i-th order* for f at $w \in \mathbb{R}^k$ if $\partial_x^i f(w, x_j) = y_j$ for all $j \in [n]$ where $x_j, y_j \in \mathbb{R}$ denote the j-th coordinates of x and y respectively.

Support points can be considered as points surpassed by the graph of $f(w, \cdot) : \mathbb{R} \to \mathbb{R}$ (see visualization on the left in Fig. 3). Typically, such support points are physically motivated and if latent properties of the process change, certain support points change their position in curve space. For instance, in a staking process, the position $x(t)$ and value $y(t)$ of the maximal force, i.e., where the first derivative is zero, starts shifting (see Fig. 2). That is, we can describe this behavior by modelling the support points $(x(t), y(t))$ and $(x(t), 0)$ of first and second order respectively, i.e. $f(w(t), x(t)) = y(t)$ and $\partial_x^1 f(w(t), x(t)) = 0$. We formalize in Sect. Data generation how we can use this to generate process curves and drifts synthetically.

Data generation

Let $f : \mathbb{R}^k \times \mathbb{R} \to \mathbb{R}$ be as in Sect. Statistical framework to model process drifts a proxy for the physical relations for given manufacturing process. In this section, we build our synthetization framwork upon the setup introduced in Sect. Statistical framework to model process drifts. Here, we neither focus on how $w(t)$ behaves in latent space, nor on how f is formulated exactly. Instead of modeling the evolution of $w(t)$ with a dynamic system, our idea is to model the behavior of support points over process executions in curve space and to seek for parameters $w(t)$ using non-linear optimization satisfying the support point conditions from Definition 2.1. For the remainder of this section, we explain how $w(t)$ can be computed given the support points. Thus, assume we have for each process execution $t \in [T]$ support points $(x^1(t), y^1(t)), \ldots (x^l(t), y^l(t))$ with with $x^i(t), y^i(t) \in \mathbb{R}^{n_i}$, that is,

$$\partial_x^i f(w(t), x_j^i(t)) = y_j^i(t) \quad \forall j \in [n_i]. \tag{3.1}$$

An overview of our notation used is shown in Fig. 4. Instead of modelling $w(t)$ explicitly, we *compute* $w(t)$ implicitly such that (3.1) is satisfied. For instance, if f is $l+2$-times differentiable in its second argument and if $\partial_w^2 \partial_x^i f$ exists, we can solve (3.1) individually for all $t \in [T]$ using second-order quasi-Newton methods (Fletcher, 1987, Chapter 3) for the objective function

$$w(t) = \arg\min_{w\in\mathbb{R}^k} \sum_{i=1}^{l} \sum_{j=1}^{n_i} D_i \cdot \left(\partial_x^i f(w, x_j^i(t)) - y_j^i(t)\right)^2$$

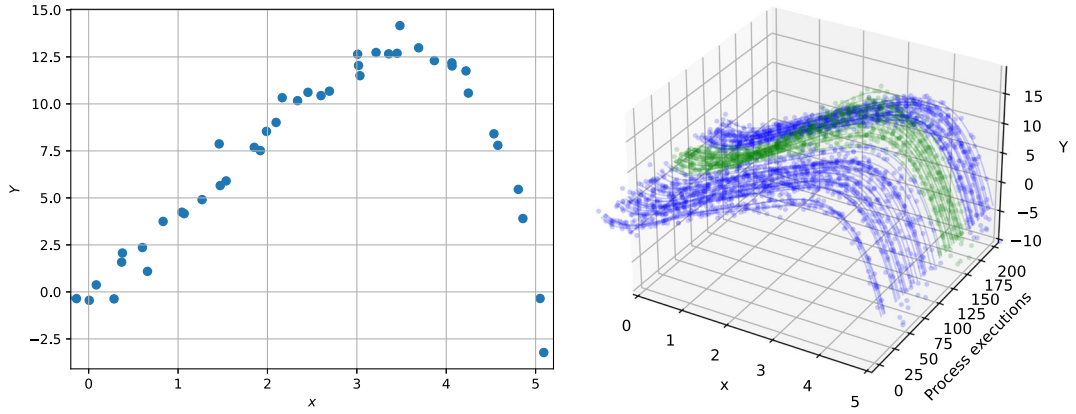


Fig. 2 Samples from a process curve (left) as well as a sequence of curve samples (right)

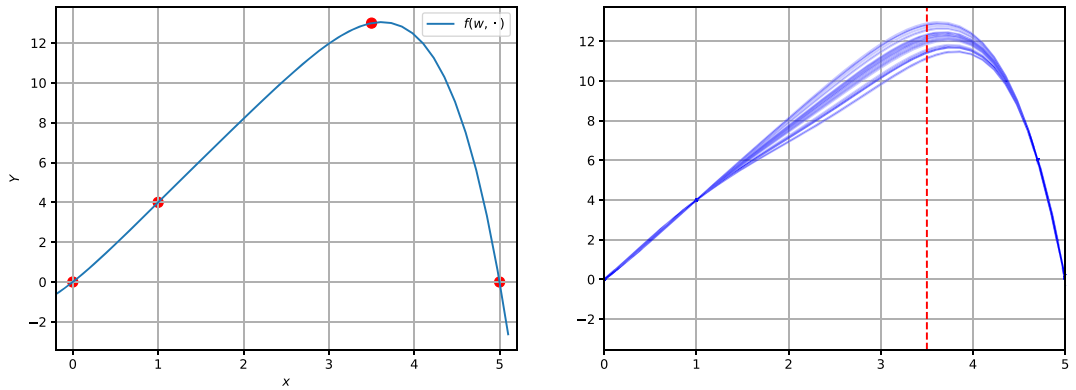


Fig. 3 Visualization of the data synthetization given a function $f(w,x) = \sum_{i=0}^{5} w_i \cdot x^i$. Left figure shows $f(w,\cdot)$ solved for concrete x^i, y^i (red points). Right figure shows sequence $f(w_1,\cdot),\ldots,f(w_{100},\cdot)$ where gaussian noise was added on one coordinate in $y^1(t)$ (corresponding coordinate $x^1(t)$ is marked with a dashed line)

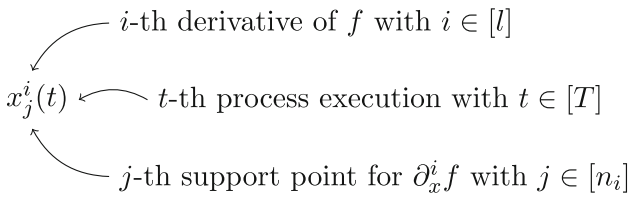


Fig. 4 Short overview of our notation

$$\left|\partial_x^i f(w(t), x)\right| \leq D_i \quad \text{for } i \in [l] \tag{3.2}$$

where $D_1,\ldots,D_l$ are constants to account for the different value ranges of the functions $\partial_x^i f$. Thus, by solving (3.2) for each $t \in [T]$, we obtain a sequence $w(1),\ldots,w(T) \in \mathbb{R}^k$ and consequently, we get a sequence of functions $f(w(1),\cdot),\ldots,f(w(t),\cdot)$. Now, these functions can be evaluated on arbitrary sets $I_t \subset \mathbb{R}$ whose points not necessarily need to be equidistant. Setting $C_t = f(w(t), I_t) + \epsilon_y \in \mathbb{R}^{|I_t|}$, we finally obtain a sequence of process curves $C_1,\ldots,C_T$. A compact overview of the data generation method is shown in Algorithm 1.

Algorithm 1 Generation of process curves

Input: $f : \mathbb{R}^k \times \mathbb{R} \to \mathbb{R}, x^i(1), y^i(1), \ldots x^i(T), y^i(T) \in \mathbb{R}^{n_i}, i \in [l]$, $\overline{x} \in \mathbb{R}, \Delta x \in \mathbb{R}_{>0}, m \in \mathbb{N}$
Output: Process curves $C_1,\ldots,C_T$

```
for t ∈ [T] do
    Compute solution w(t) for (3.2) using support points (x^1(t), y^1(t)), ..., (x^l(t), x^l(t))
    I_t ← {x̄ + j · Δx + ε_x : j ∈ [m]}
    C_t ← f(w(t), I_t) + ε_y
end for
return C_1, ..., C_T
```

It is left to show how to generate the support points as input for Algorithm 1. One way is to use support points of a real process curve dataset, and using Algorithm 1 to create a semi-synthetic copy of it. In a fully synthetic setting, the support points at execution $t \in [T]$, the support points $(x^i(t), y^i(t))$ can be sampled from a distribution on $\mathbb{R}^{n_i}$ respectively, whose statistical properties change over the temporal axis. For instance, $y^i(t) \sim \mathcal{N}_{\mu^i(t),\sigma}$ with $\mu^i : [T] \to \mathbb{R}^{n_i}$ encoding the drift behavior over the temporal axis for the support

points. In Appendix B, we showcase in depth an application of Algorithm 1 where f is a polynomial.

Remark 3.1 (Multivariate data) Our theoretic framework extends naturally to multivariate time series data, where each dimension $d \in [c]$ (or signal) has its own function f_d. If they do not share their latent information $w_d(t)$, then Algorithm 1 can be executed for each dimension individually. If they share some latent information, then (3.2) can be extended by summing all support point conditions for all $f_1, \ldots, f_c$.

Remark 3.2 (Profile data) Our theoretic framework is also capable to generate profile data with drifts holding both, drifts within a profile and drifts over executions. This can be obtained, for instance, by describing how the support points should behave in each profile and for subsequent profiles.

Remark 3.3 (Choosing f) The function f is a free and important parameter of Algorithm 1 and impacts its performance. Which function class to select depends on the number and order of the support points. In our examples (see also Sect. Data generation with polynomials), a polynomial whose degree arises from the number of support points to fit suffices. In principle, f can be chosen from any parametrized function class, like B-splines, Gaussian processes (Rasmussen, 2004), neural networks (Hornik et al., 1989), or Kolmogorov-Arnold networks (Liu et al., 2025). For overparametrized function classes, regualization-terms for $w \in \mathbb{R}^k$ can be added to the objective in Eq. (3.2). Algorithm 1 can also be used to generate synthetic copies of a real datasets of curves $C_1, \ldots, C_T$. Here, all values given in C_t can be considered as support points of 0-th order for the t-th execution and a good function class for f can be found via model selection by minimizing fitting errors in Eq. (3.2). In case only a few real curve samples exist, a degradation pattern to be interpolated between selected support points may have to be learned first along the temporal axis.

The temporal area under the curve

Different usecases require different performance metrics to evaluate algorithms. In classification, for instance, sometimes avoiding false positives is, sometimes avoiding false negatives. However, when it comes to general benchmarking classifiers somewhat independently of their precise application in the sense to see how well their response correlates to the actual class label, the AUC (Hanley & McNeil, 1982) is frequently used. However, the vanilla AUC takes samples independently of their temporal context, that is, independent of samples from the previous and next process execution. Thus, we construct in this section a more suitable metric to measure the predictive power of machine learning models for process drift detection. In order to do so, we first formalize what we understand as a process drift and which assumptions we require. Let $C_1, \ldots, C_T$ be a sequence of process curves and let $\mathcal{D} \subset [T]$ be the set of curve indices belonging to drifts. Our first assumption is that drifts, different from point anomalies, appear sequentially and can be uniquely decomposed into disjoint segments:

Definition 4.1 (*Drift segments*) Let $\mathcal{D} \subset [T]$. Then a series of subsets $\mathcal{D}_1, \ldots, \mathcal{D}_k \subset \mathcal{D}$ is a *partition of drift segments* if there exists $1 \leq l_1 < h_1 < l_2 < h_2 < \ldots, < l_k < h_k \leq T$ such that for all i, we have $\mathcal{D}_i = [l_i, h_i]$ and $\mathcal{D} = \cup_{i=1}^{k} \mathcal{D}_i$.

The drift segments can be considered as a partition of the smallest consecutive drifts which cannot decomposed any further into smaller segments. Now, assume we also have the output $s \in \mathbb{R}^T$ of a detector where each coordinate s_t quantifies how likely the curve C_t of the t-t-h process execution belongs to a drift, that is, the higher s_t the more likely the detector classifies $t \in \mathcal{D}$ (see also Fig. 5). By choosing a threshold $\tau \in \mathbb{R}$, we can construct a set

$$\hat{\mathcal{D}}(s, \tau) := \{t \in [T] : s_t \geq \tau\}$$

which serves as a possible candidate for $\mathcal{D}$. Clearly, if $\tau_1 \geq \tau_2$, then $\hat{\mathcal{D}}(s, \tau_1) \subseteq \hat{\mathcal{D}}(s, \tau_2)$. Its also straight-forward to see that for every τ, the set $\hat{\mathcal{D}}(s, \tau)$ decomposes uniquely into drift segments $\hat{\mathcal{D}}_1, \ldots, \hat{\mathcal{D}}_l$ as defined in Definition 4.1 and that the length and number of these atomic segments depends on τ. Now, to quantify the predictive power of the detector yielding s, one needs to quantify how *close* $\hat{\mathcal{D}}(s, \cdot)$ is to $\mathcal{D}$ when τ varies. There are many established set-theoretic measurements that are widely used in practice to quantify the distance between two finite and binary sets A and B, like the Jaccard index $\frac{|A \cap B|}{|A \cup B|}$, the Hamming distance $|A \backslash B| + |B \backslash A|$, or the Overlap coefficient $\frac{|A \cap B|}{\min(|A|, |B|)}$ just to name a few. Most metrics, however, have as a build-in assumption that the elements of the set are iid and hence the temporal context is largely ignored making them unsuitable for process drift detection. Moreover, for most detectors we have to select a discrimination threshold τ, making evaluation cumbersome as it requires to tune the threshold on a separate held-out dataset. Moreover, in most practical scenarios, $\mathcal{D}$ is only a small subset and thus the evaluation metric has to consider highly imbalanced scenarios as well.

Clearly, detectors are required where all true drift segments $\mathcal{D}_i$ are overlapped by predicted drift segments. For this, let $L_i := \{j \in [l] : \mathcal{D}_i \cap \hat{\mathcal{D}}_j \neq \emptyset\}$. Clearly, $L_i \cap L_{i+1} \neq \emptyset$ if $\mathcal{D}_i$ and $\mathcal{D}_{i+1}$ both intersect with a predicted drift segment. Now, the set $\mathcal{T}_i := \cup_{j \in L_i} \hat{\mathcal{D}}_j$ which is the union of all predictive segments intersecting with $\mathcal{D}_i$ serves as a candidate for $\mathcal{D}_i$. To measure how well $\mathcal{D}_i$ is covered - or overlapped - by $\mathcal{T}_i$ we define the *soft overlap score* inspired by the Overlap

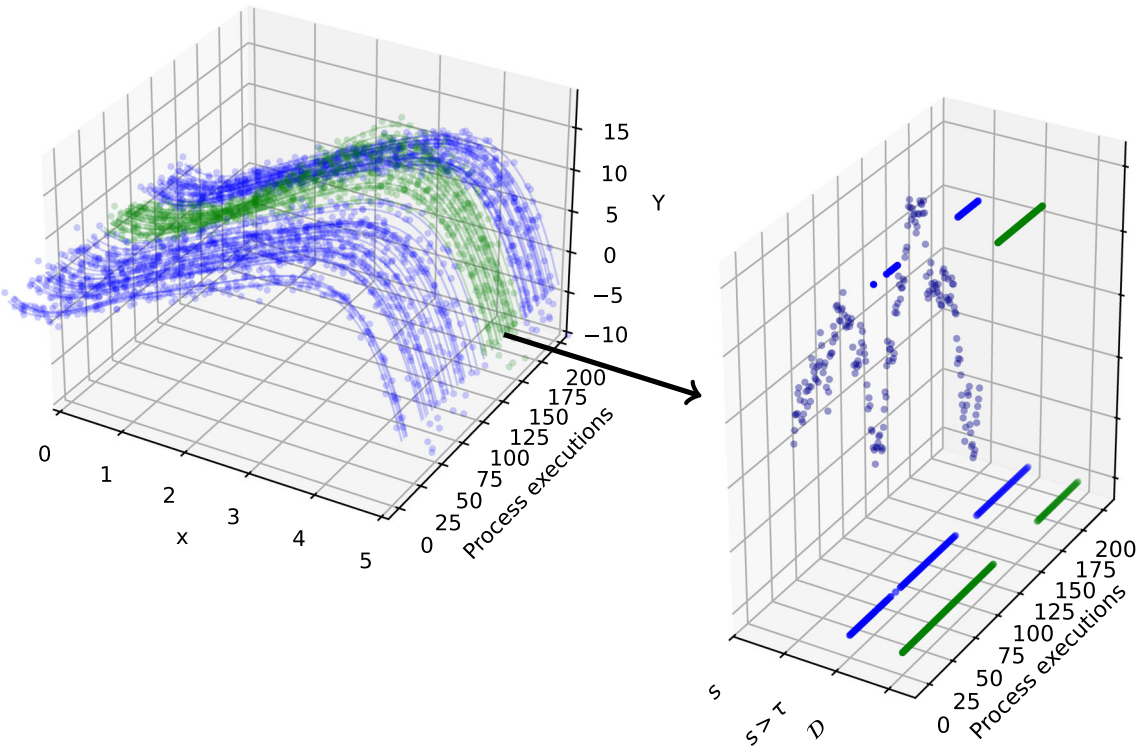


Fig. 5 Applying a process drift detector on each process curves yields a score s which needs to be compared to the ground truth $\mathcal{D}$ for each threshold τ

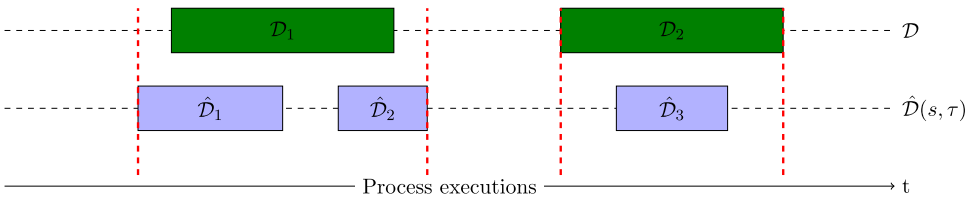


Fig. 6 Temporal arrangements of true and predicted drift segments as input for Algorithm 2

coefficient as follows:

$$\text{sOLS}(\mathcal{D}_i, s, \tau) := \frac{|\mathcal{T}_i|}{\max(\mathcal{T}_i \cup \mathcal{D}_i) - \min(\mathcal{T}_i \cup \mathcal{D}_i) + 1} \tag{4.1}$$

Obviously, an sOLS of 1 is the best possible and this is reached if and only if $\mathcal{T}_i = \mathcal{D}_i$. It is easy to see that for fixed $\mathcal{D}_i$, the enlargement of $\mathcal{T}_i$ beyond the boundaries of $\mathcal{D}_i$ improves the overlap score, as $|\mathcal{T}_i|$ increases and one of either $\max(\mathcal{T}_i \cup \mathcal{D}_i)$ or $-\min(T_i \cup \mathcal{D}_i)$ increases as well. A special case is if $\mathcal{D}_i$ is completely covered by $\mathcal{T}_i$, i.e. $\mathcal{D}_i \subseteq \mathcal{T}_i$, then it follows that $\mathcal{T}_i$ is an interval as well and thus $\text{sOLS}(\mathcal{D}_i, s, \tau) = 1$. When $\mathcal{T}_i$ enlarges, then the number of false positives, i.e. the time points t contained in some $\hat{\mathcal{D}}_i$ and in the complement $\overline{\mathcal{D}} := [T] \setminus \mathcal{D}$ of the ground truth $\mathcal{D}$, enlarges as well. Thus, the predictive power of a detector is shown in the overlap score as well as the created false positive rate

$$\text{FPR}(\mathcal{D}, s, \tau) := \frac{|\hat{\mathcal{D}}(s,\tau) \cap \overline{\mathcal{D}}|}{|\overline{\mathcal{D}}|}$$

into account. To also take false negatives into account, the enumerator in (4.1) could be changed as follows, yielding our final definition of the *overlap score*:

$$\text{OLS}(\mathcal{D}_i, s, \tau) := \frac{|\mathcal{T}_i \cap \mathcal{D}_i|}{\max(\mathcal{T}_i \cup \mathcal{D}_i) - \min(\mathcal{T}_i \cup \mathcal{D}_i) + 1}. \tag{4.2}$$

Algorithm 2 illustrates in detail how the *Overlap score* $\text{OLS}(\mathcal{D}, s, \tau)$ can be computed algorithmically.

Our score considers both, the OLS and the FPR, which mutually influence each other. In the computation of the AUC, any threshold τ from $[\min_t(s_t), \max_t(s_t)]$ yields a pair of false positive rate $\text{FPR}(\mathcal{D}, s, \tau)$ and true positive rate $\text{TPR}(\mathcal{D}, s, \tau)$ which can be drawn as a curve in the space

Algorithm 2 Overlap score OLS($\mathcal{D}, s, \tau$)

Input: $\mathcal{D} \subset [T]$, $s \in \mathbb{R}^T, \tau \in \mathbb{R}$
Output: Overlap score
$\mathcal{D}_1, \ldots, \mathcal{D}_k \leftarrow$ find drift segments of $\mathcal{D}$
$\hat{\mathcal{D}}_1, \ldots, \hat{\mathcal{D}}_l \leftarrow$ find drift segments of $\hat{\mathcal{D}}(s, \tau)$
$o \leftarrow \mathbf{0} \in \mathbb{R}^k$
for $i \in [k]$ **do**
  $L_i \leftarrow \{j \in [l] : \hat{\mathcal{D}}_j \cap \mathcal{D}_i \neq \emptyset\}$ ▷ All predicted drift segments overlapping with $\mathcal{D}_i$
  $\mathcal{T}_i \leftarrow \cup_{j \in L_i} \hat{\mathcal{D}}_j$ ▷ Union of all segments intersecting with $\mathcal{D}_i$
  $o_i \leftarrow \frac{|\mathcal{T}_i \cap \mathcal{D}_i|}{\max(\mathcal{T}_i \cup \mathcal{D}_i) - \min(\mathcal{T}_i \cup \mathcal{D}_i) + 1}$ ▷ fraction of overlap
end for
return $\frac{1}{k} \sum_{i=1}^{k} o_i$

where FPR is on the x-axis and TPR on the y-axis. Similarly, we define the *temporal area under the curve*, or just TAUC, as the area under the FPR-OLS curve while the discrimination threshold τ varies. We refer to the *soft* TAUC, or just sTAUC, to the area under the FPR-sOLS curve (see Fig. 7).

Note that the integral of the curve can be computed using two different methods, the *step rule* and the *trapezoidal rule* and depending on which method is used, the value of the score may differ. We showcase this behavior in detail for trivial detectors in Appendix D. In Appendix C, we investigate in several synthetic cases in depth the differences and similarities between sTAUC, TAUC, and AUC.

Example 4.2 Consider the situation shown in Fig. 6. There, we have two true drift segments $\mathcal{D}_1$ and $\mathcal{D}_2$, and three segments $\hat{\mathcal{D}}_1$, $\hat{\mathcal{D}}_2$ and $\hat{\mathcal{D}}_3$ as drift segments of some detector output $\hat{\mathcal{D}}(s, \tau)$. Clearly, $L_1 = \{1, 2\}$ where $L_2 = \{3\}$ as only $\mathcal{D}_3$ overlaps with $\mathcal{D}_2$. To unveil $\mathcal{D}_1$, the detector needs to separate drift segments, leading to false negatives and positives and thus a relatively small OLS. On the other hand, as $\hat{\mathcal{D}}_3 \subset \mathcal{D}_2$, we have $\mathcal{T}_2 = \hat{\mathcal{D}}_3$.

Experiments

Next, we benchmark existing algorithms on data generated with our framework `driftbench` and reporting the TAUC. All datasets and algorithms used are available in the repository of `driftbench`. The goal of the benchmark is to provide a proof of concept for our score and data generation method, not to be very comprehensive on the model side. Thus, based on our literature research in Sect. Introduction we have hand-selected a small set of typically used model patterns drift detectors used in practice consists of (see Sect. Algorithms).

The basic evaluation loop follows a typical situation from manufacturing, where process engineers have to identify time periods within a larger curve datasets where the process has drifted. Thus, all models consume as input a process curve dataset $C_1, \ldots, C_T$ and do not have access to the ground truth $\mathcal{D}$, which is the set of curves belonging to a drift (see Sect. Datasets). Afterwards, each model predicts for each curve C_t from this dataset a score $s_t \in \mathbb{R}$, and afterwards, the TAUC, sTAUC, and AUC are computed for $s = (s_1, \ldots, s_T)$. To account for robustness, we generate each dataset of a predefined specification five times for a different random seed each, leading to slightly different datasets of roughly same complexity. All models are trained unsupervised, i.e. without any information of the true drift segments.

Algorithms

The algorithms used can be decomposed into multiple steps (see also Fig. 8), but not all algorithms use all steps. First, there are may some features extracted from each curve. Afterwards, a sliding window collects and may aggregate these such that a score is computed.

Feature extraction

In this step, we use autoencoders (Rumelhart et al., 1986) to compute a k-dimensional representation $e_t \in \mathbb{R}^k$ for each high-dimensional process curve C_t with k small. The indention behind is to estimate an inverse of the unknown function f and to recover information about the support points used. Moreover, we also apply deterministic aggregations over the x-information of each curve C_t.

Windowing and aggregation

In this step, the algorithms may aggregate the data from the previous step using a fixed window of size m that is applied in a rolling fashion along the process iterations. One aggregation we use is to first compute for each coordinate $j \in [k]$ of $e_t \in \mathbb{R}^k$ with $t \geq m$ the rolling mean $a_{t,j} = \frac{1}{m} \sum_{i=t-m+1}^{t} e_{i,j}$. These values can then further be statistically aggregated, like by taking the maximum $a_t := \max\{a_{t,j} : j \in [k]\}$.

Score computing

Goal of this step is to compute a threshold which correlates with the ground truth, that is, the larger the higher the possibility of a drift. Here, we may also aggregate previous features in a rolling fashion. The simplest aggregation we use is to compute the euclidean distance of subsequent elements $s_t = \|a_t - a_{t-1}\|_2$ which is just the absolute difference if a_t and a_{t-1} are scalars. If a_t is a scalar, we also can compute the rolling standard deviation, again over a window of size

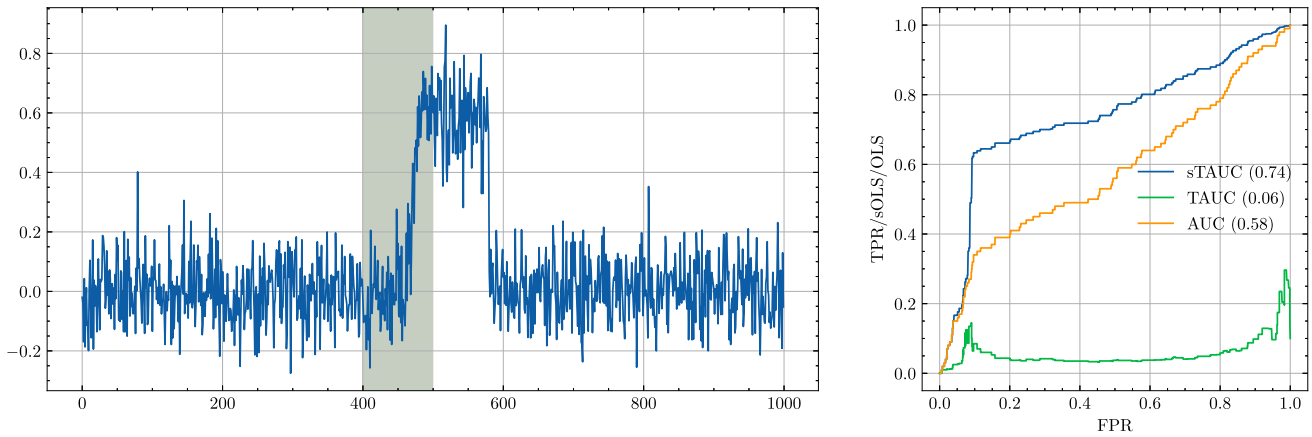


Fig. 7 The TPR, sOLS, and OLS when the FPR varies for the synthetic prediction on the left

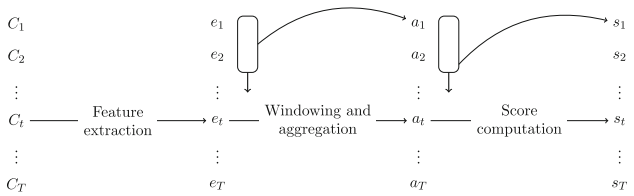


Fig. 8 A high-level overview of the elementary tasks of the detectors used

m, like this:

$$s_t = \sqrt{\frac{1}{m-1}\sum_{j=t-m+1}^{t}\left(a_j - \left(\frac{1}{m}\sum_{i=t-m+1}^{t} a_i\right)\right)^2}.$$

Another approach follows a probabilistic path by testing if a set of subsequent datapoints $\{a_{t-m+1}, \ldots, a_t\}$ come from the same distribution as a given reference set. In our study, we use a windowed version (Raab et al., 2020) of the popular Kolmogorov-Smirnov test (Simard & L'Ecuyer, 2011), often called KSWIN, which makes no assumption of the underlying data distribution. However, this can only be applied when a_t is a scalar. More particularly, we define two window sizes, m_r for the reference data and m_o for the observation. The windows are offset by constant $\delta > 0$. We then invoke the KS-test and receive a p-value p_t, which is small if the datasets come from different distributions. Thus, one way to derive a final score is to compute $s_t = \log(1 + \frac{1}{p_t})$. Another probabilistic method we use in our study based on a multivariate statistical test is the *Maximum Mean Discrepancy* (MMD) (Gretton et al., 2012). This method uses feature mappings based on kernels, and calculates the distance between the means in these mappings. MMD also makes no assumption about the underlying distribution, and works on multidimensional data. We use this method in the same way using two windows as described in the KS-test. We also evaluate algorithms that derive their score based on a similarity search within $\{a_1, \ldots, a_t\}$. Here, we use clustering algorithms, like the popular k-means algorithm, and use the euclidean distance to the computed cluster center of a_t as s_t. Another way is to fit a probability density function on s_t, like a mixture of Gaussian distributions, and to set s_t as the log likelihood of a_t within this model.

Algorithm overview

Here is a short summary of the algorithms used in our benchmark study:

- `RollingMeanDifference`(m_r): First, the rolling mean over a window of size m_r is computed over all values for the respective curves in the window. Afterwards, the maximum value for each curve is taken and the absolute difference between two consecutive maximum values is computed.
- `RollingMeanStandardDeviation`(m_r): First, the rolling mean over a window of size m_r is computed over all values for the respective curves in the window. We also choose the maximum value of these computed values per curve. Then, we compute the standard deviation using the same window for this one-dimensional input.
- `SlidingKSWIN`(m_r, m_o, δ): We compute the mean value for each curve and apply a sliding KS-test on this aggregated data. We use two windows of size m_r and m_o where the windows are offset by δ.

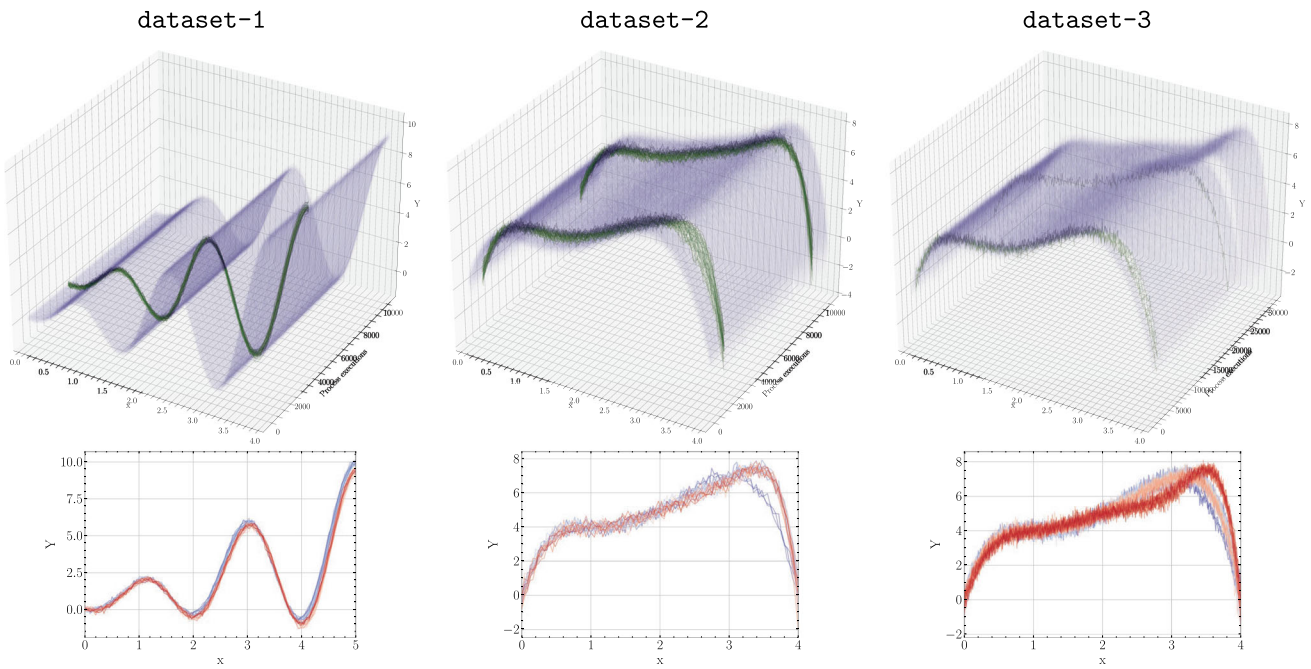


Fig. 9 The datasets used in our benchmark study. The true drift segments are marked in green. Lower figures show selected curves, whose color encodes the process iteration $t \in [T]$ - blue marks smaller t values, red larger ones. Recall that `dataset-`k has k many drift segments (Color figure online)

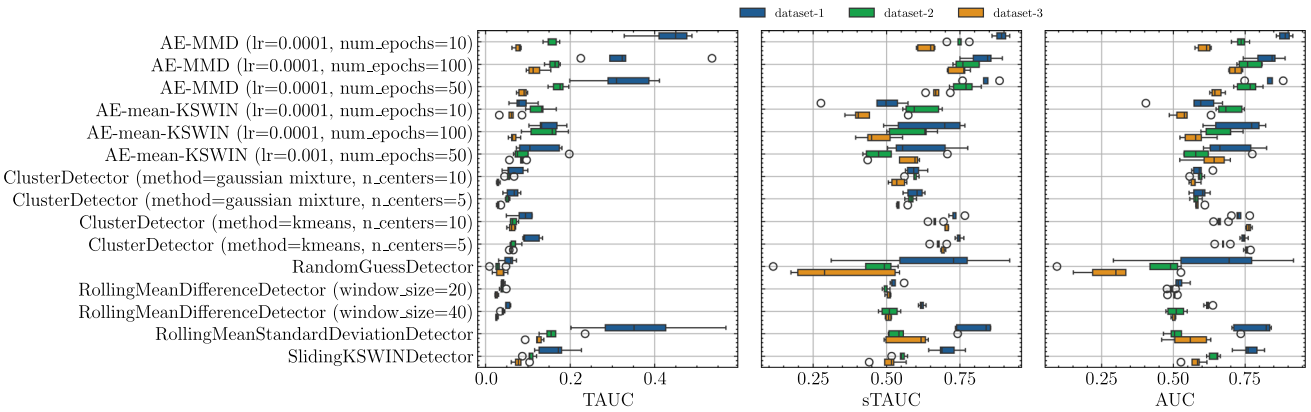


Fig. 10 Benchmark results on `dataset-1`, `dataset-2`, and `dataset-3`

- `Cluster`(n_c): A cluster algorithm performed on the raw curves using n_c clusters where score is distance to the closest cluster center.
- `AE`(k)`-mean-KS`(m_r, m_o, δ): First, an autoencoder is applied extracting computing k many latent dimensions. Afterwards, the mean across all k latent dimensions is computed. Finally, a sliding KS-test is applied with two windows of sizes m_r and m_o, where the windows are offset by δ.
- `AE`(k)`-MMD`(m_r, m_o, δ): First, an autoencoder is applied extracting computing k many latent dimensions. Afterwards, a k-dimensional sliding MMD-test is applied with two windows of sizes m_r and m_o, where the windows are offset by δ.

Datasets

We benchmark the algorithms listed in 5.1 on three different datasets (see Fig. 9) created with our framework `driftbench`, all designed to comprise different inherent challenges.

The datasets `dataset-2` and `dataset-3` have been inspired by the force signals of staking processes (see also Göbler et al., 2024, A.1) where we used $f(w, x) = \sum_{i=0}^{7} w_i \cdot x^i$ as function to generate them. The `dataset-2` consists of $T = 10.000$ curves, each called on $|I| = 100$ equidistant values between $[0, 4]$, i.e., $\overline{x} = 0$ and $\Delta x = 0.04$.

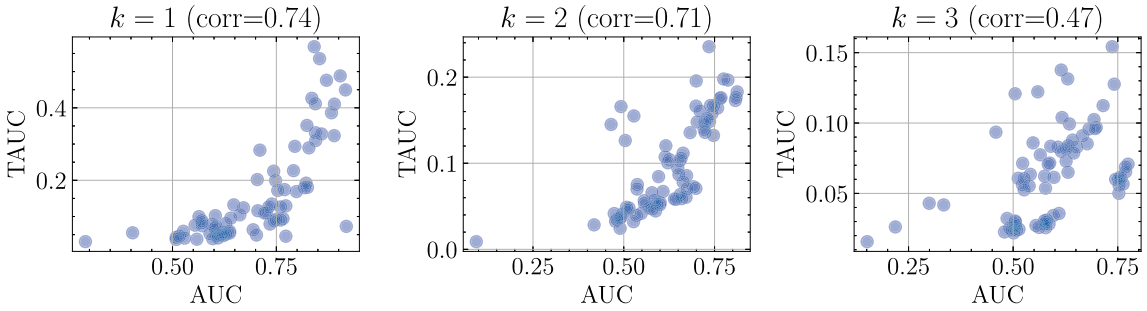


Fig. 11 Correlation between TAUC and AUC for different number of true drift segments k

On the other hand, `dataset-3` consists of $T = 30.000$ curves each having $|I| = 400$ values between $[0, 4]$. Both datasets have drifts that concern a movement of the global maximum together with drifts where only information of first order changes over time. In the generation process of `dataset-1`, we used $f(w, x) = w_0 \cdot x \cdot \sin(\pi \cdot x - w_1) + w_2 \cdot x$ and generated $T = 10.000$ many curves, each having $|I| = 100$ datapoints. It only holds a single drift, where the global minimum at drifts consistently over a small period of time along the x-axis. In all datasets, the relative number of curves belonging to a drift is very small: roughly 1 percent in `dataset-1`, 2 percent in `dataset-2`, and 0.1 percent in `dataset-3`. Particularly, `dataset-`k has k many drift segments. To generate a drift segment $[t_0, t_1]$ for a given support point where the value should change linearly from a to b (see also Sect. Data generation with polynomials), we sampled from normal distributions $\mathcal{N}_{\mu(t),\sigma}$ with fixed σ and mean

$$\mu(t) = \begin{cases} a, & \text{if } t < t_0 \\ b \cdot \frac{t - t_0}{t_1 - t_0} + a, & \text{if } t_0 \leq t \leq t_1 \\ b, & \text{if } t_1 < t \end{cases}.$$

Results

The result of our benchmark study is shown in Fig. 10. Generally, there is a discrepancy in detectors of the highest AUC and the highest TAUC. More concrete, the larger the number of true drift segments in a dataset is, the larger the discrepancy (see also Fig. 11). For instance, the `RandomGuessDetector` reached the highest AUC score on `dataset-1`, where it ranges on all three datasets among the last ranks in the TAUC score. On all datasets, autoencoder-based systems reach among the best detectors for both, TAUC and AUC. Those using a multivariate test in their latent space reach better scores than these using an aggregation of multiple uni-variate tests.

Although some cluster-based systems archive good AUC scores on dataset `dataset-3`, none of the benchmarked algorithms is capable to compute a score that can be used to recover the true drift segments, resulting in small TAUC scores for all algorithms (see also Fig. 14). The respective predictions over the temporal dimension of the best detectors are shown in Appendix A in more detail, where it also becomes visible that detectors with higher TAUC better recover the true drift segments.

Conclusion

This work shows how algorithms designed to detect process drifts can be benchmarked in robust and reliable way. We have introduced a scalable and controllable data generation method that creates process curves datasets with drifts and a verified ground truth. In our approach, process curve datasets can be solely generated by modelling the behavior of support points over the temporal axis and using non-linear optimization. We then introduce and study the novel *TAUC* score which is particularly designed to evaluate the performance of drift detectors on their temporal consistency over sequential process executions. We proved the effectiveness of our approach in a small benchmark study. Our results reveal that existing algorithms often struggle with datasets containing multiple drift segments, underscoring the need for further research.

Appendix A: Predictions of detectors of benchmark study

Figures 12, 13, and 14 show the prediction $s \in \mathbb{R}^T$ of the detectors reaching highest TAUC and AUC scores on the individual datasets are shown respectively.

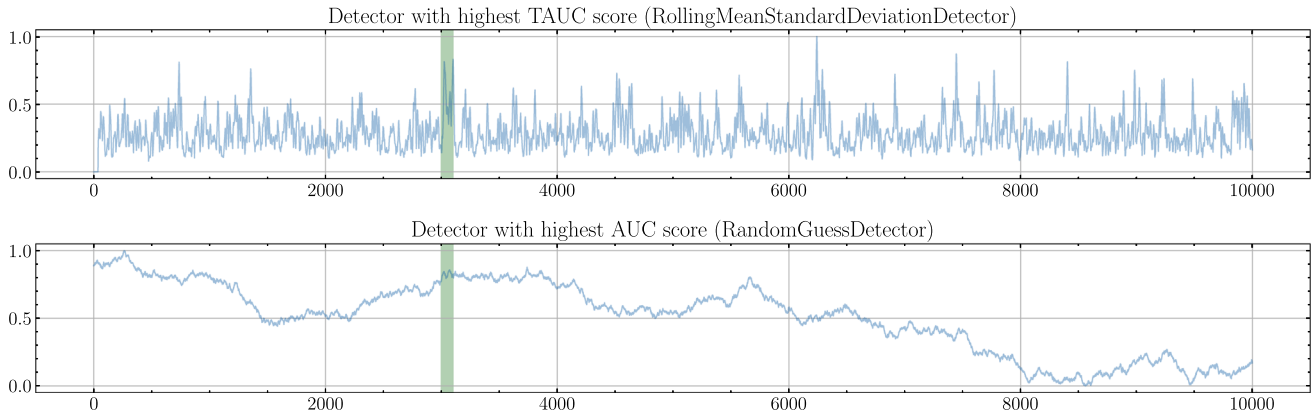


Fig. 12 Best detectors on `dataset-1`

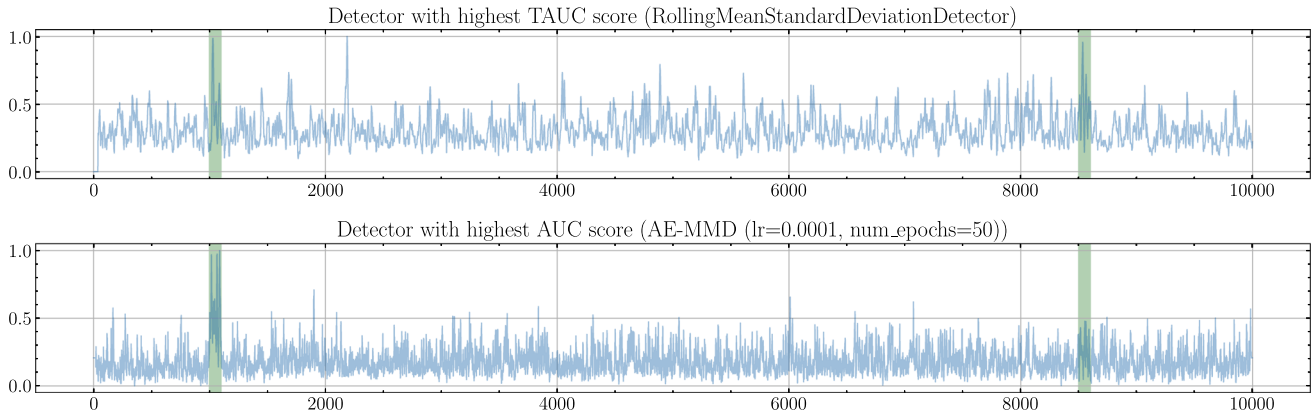


Fig. 13 Best detectors on `dataset-2`

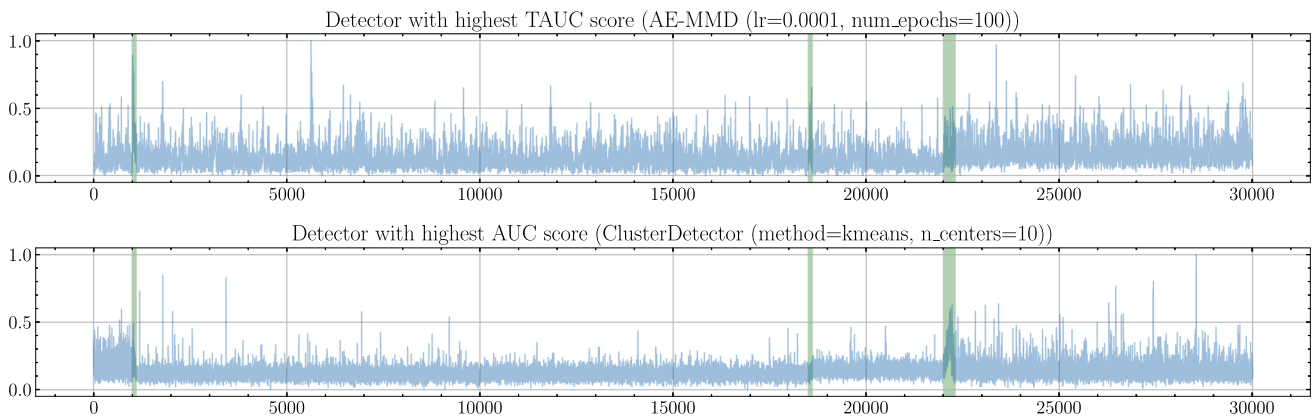


Fig. 14 Best detectors on `dataset-3`

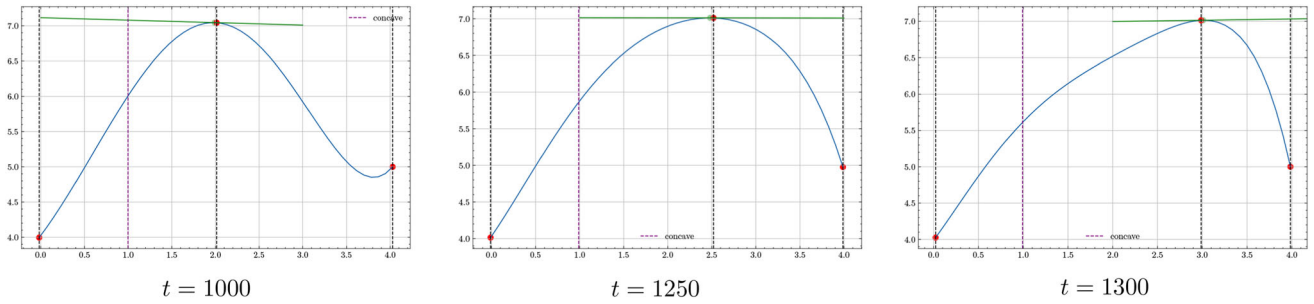


Fig. 15 Visualization of some process curves in the example dataset. The red dots indicate support points with first order information given. The green line visualizes the slope at the green dot, encoded by the condition for the first derivative. The purple dashed line indicates the curvature at the corresponding x-value, encoded by the condition for the second derivative. From $t = 1000$ to $t = 1300$, the x-value of the maximum moves from 2 to 3 (Color figure online)

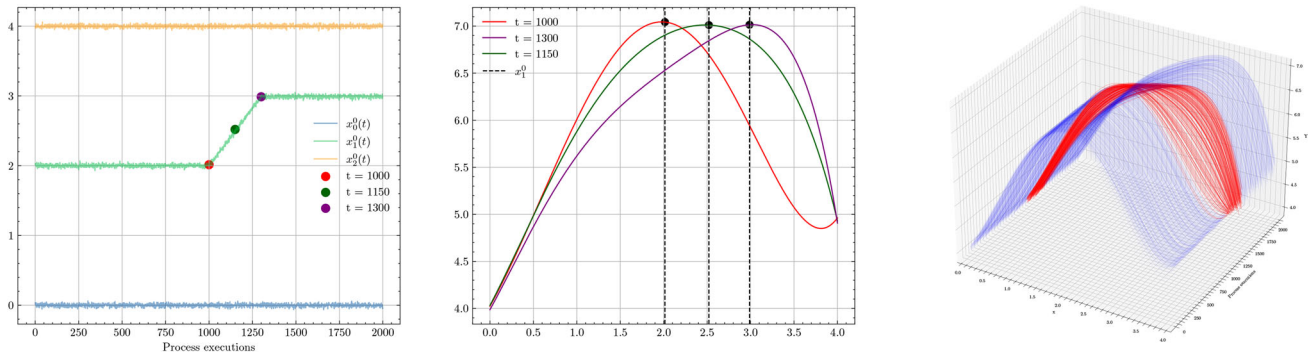


Fig. 16 Visualization of the drift applied on x_1^0 in this example, with respective curves. The left figure shows how the x_i^0 values change over time. Only x_1^0 changes, as by our drift definition from the process executions $t = 1000$ until $t = 1300$ linearly from 2 to 3, the others remain unchanged. The middle figure shows the respective curves, color-coded to the dots in the left figure

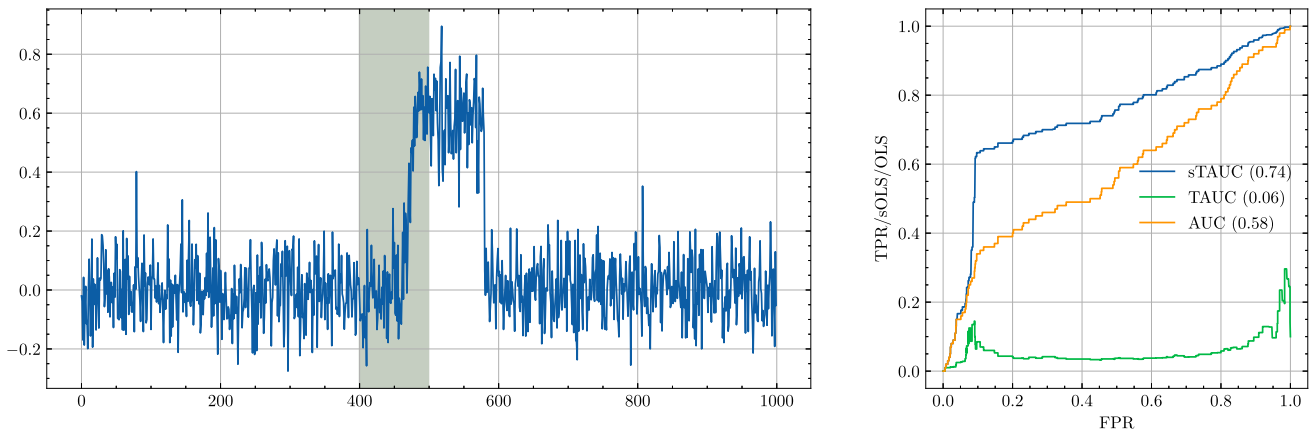


Fig. 17 Prediction of a detector that lags behind the ground truth (left) and its curves underneath the TAUC and AUC (right)

Appendix B: Data generation with polynomials

In this section, we demonstrate the data generation method introduced in Sect. Data generation along an example involving a polynomial $f : \mathbb{R}^6 \times \mathbb{R} \to \mathbb{R}$ of degree five, i.e.

$$f(w, x) = \sum_{i=0}^{5} w_i \cdot x^i$$

with $w \in \mathbb{R}^6$. We simulate 2000 process executions and thus sample 2000 process curves. The shape of each curve is defined by its support points. We are only interested in its

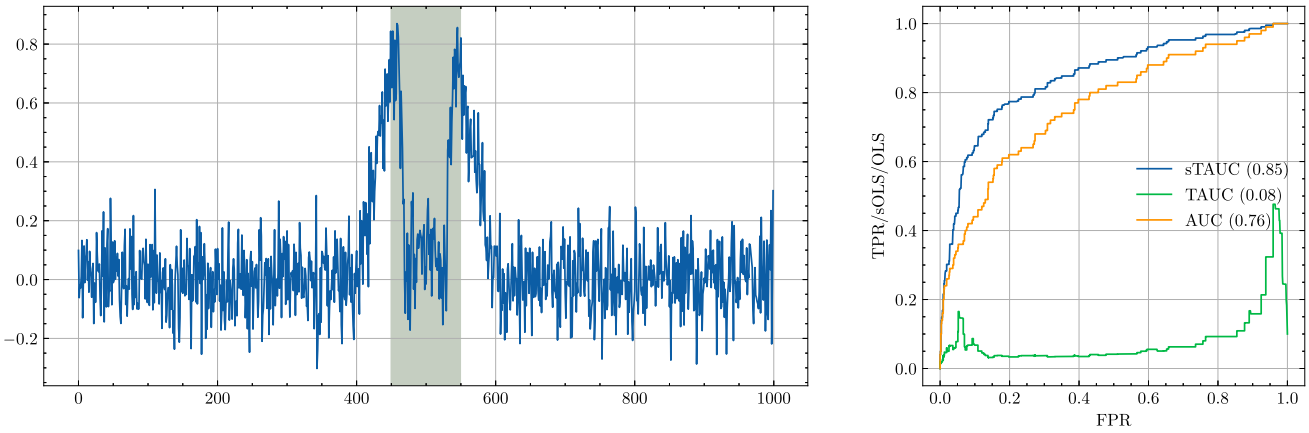


Fig. 18 Prediction of a detector that shows high scores at the boundary of the true drift segment only (left) and its curves underneath the TAUC and AUC (right)

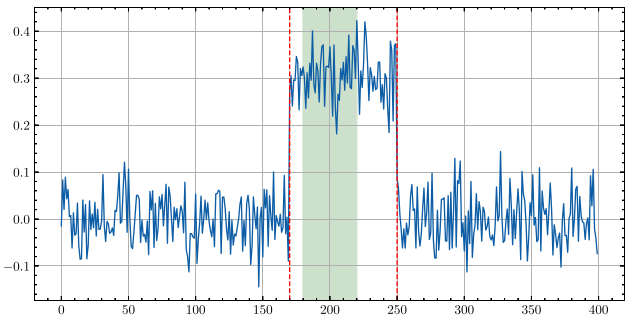


Fig. 19 Situation with single predicted segment (red dashed) and single true segment (green area) (Color figure online)

curvature in $I = [0, 4]$. First, we want to add a condition onto the start and end of the interval, namely that $f(w, 0) = 4$ and $f(w, 4) = 5$. Moreover, we would like to have a global maximum at $x = 2$, which means the first order derivative

$$\partial_x^1 f(w, 2) = \sum_{i=1}^{4} i \cdot w_i \cdot 2^{i-1}$$

should be zero and its second order derivate

$$\partial_x^2 f(w, 2) = \sum_{i=1}^{3} i \cdot (i-1) \cdot w_i \cdot 2^{i-2}$$

should be smaller than zero. Here, we want it to be -1. Finally, we want to the curve to be concave at around $x = -1$. All in all, these conditions result into the following equations, some of them are visualized in Fig. 15:

$$\partial_x^0 f(w, 2) = 7 \quad \partial_x^1 f(w, 2) = 0 \quad \partial_x^2 f(w, 2) = -1$$
$$\partial_x^0 f(w, 0) = 4 \quad \partial_x^0 f(w, 4) = 5 \quad \partial_x^2 f(w, 1) = -1$$

Then, we let the data drift at some particular features. We simulate a scenario, where the peak at x_1^0 and x_0^1 moves from the x-position 2 to 3 during the process executions $t = 1000$ until $t = 1300$. Thus, we let x_1^0 and x_0^1 drift from 2 to 3, resulting in a change of position of the peak. We let the corresponding y-values $y_1^0 = 7$ and $y_0^1 = 0$ unchanged. Now, we can solve each of the 2000 optimization problems, which results in 2000 sets of coefficients for each process curve, such that the conditions are satisfied. By evaluating f with the retrieved coefficients in our region of interest $[0, 4]$, we get 2000 synthesized process curves with a drift present at our defined drift segment from $t = 1000$ until $t = 1300$ (Fig. 16).

Appendix C: TAUC vs AUC

In this section we explore in depth the similarities and differences of the TAUC introduced in Sect. The temporal area under the curve and the established AUC. This is done along synthetic predictions.

Lagged prediction

The first example we look at is a typical scenario that appears if window-based approaches are used, namely that the prediction lags a bit behind of the true window, but still the detector overlaps a significant proportion of the drift segment (see Fig. 17). Other than the TPR, the sOLS rewards these predictors and thus the sTAUC shows a larger value than the AUC.

Change point detection

Another typical scenario is that a detector shows significantly large values at the start and end of the true drift segment, but

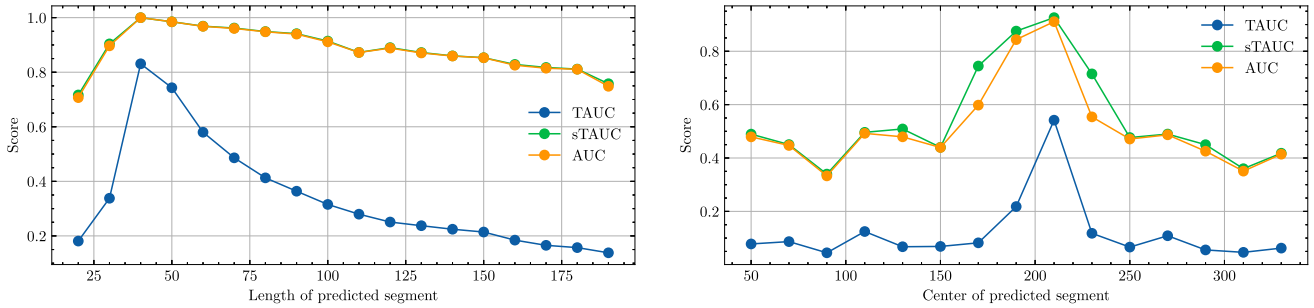


Fig. 20 Behavior of sTAUC, TAUC, and AUC when length and position of predicted segment varies

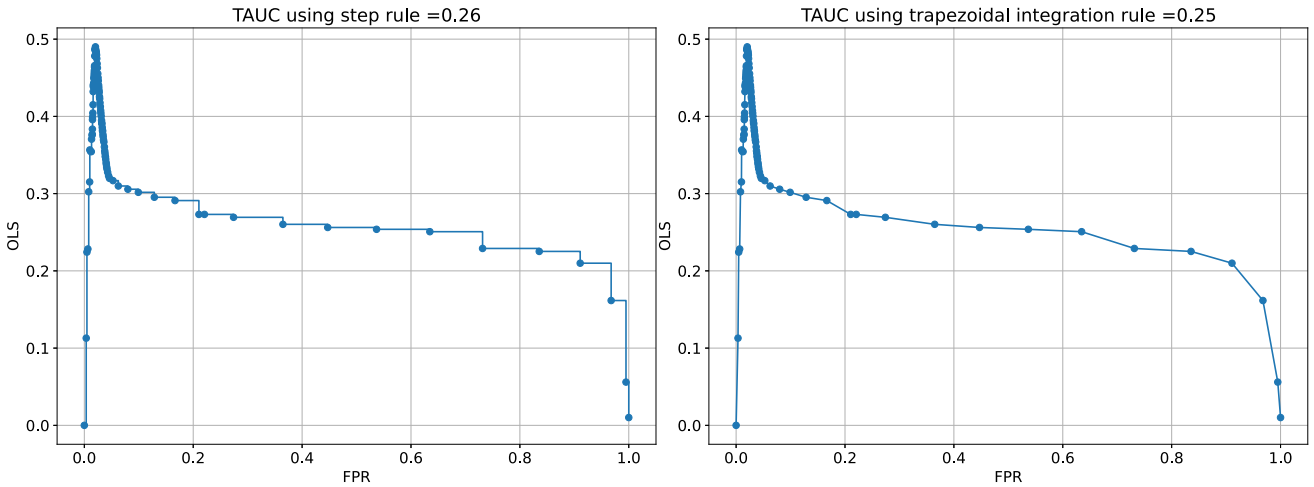


Fig. 21 Visualization of a concrete curve used to calculate the TAUC with its TAUC-score. Left figure shows the constructed curve when using the step rule, while the right figure shows the curve when calculating the TAUC using the trapezoidal integration rule

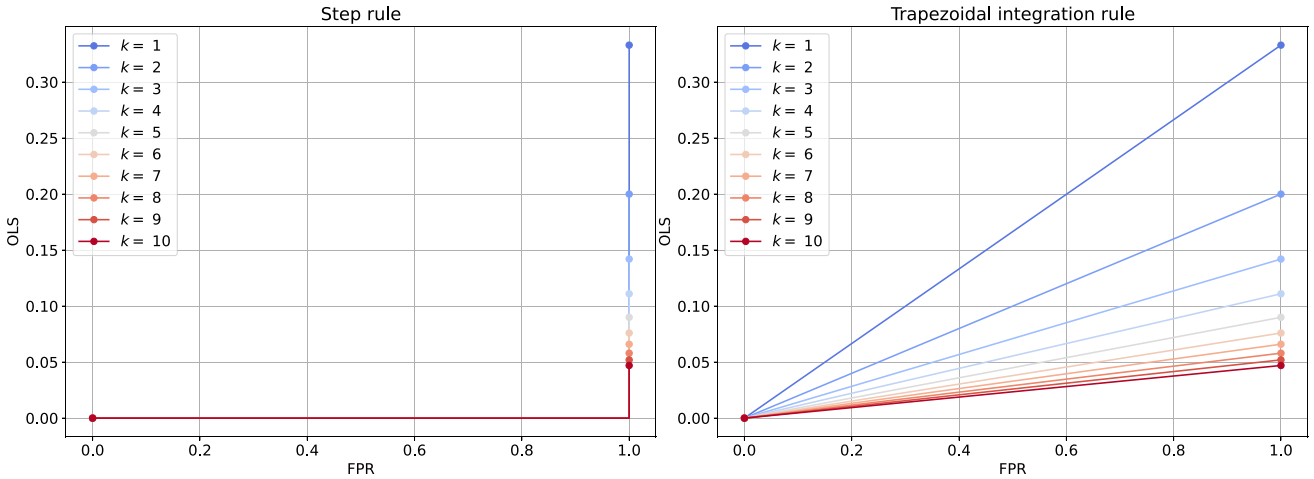


Fig. 22 Visualization of the behaviour of the constructed curve for the TAUC on increasing number of segments k. The left figure shows that the TAUC for the computation with the step rule always remains 0. The right figure shows that the area under the line decreases with increasing k, resulting in a lower TAUC value in case of the trapezoidal integration rule

sag in between (see Fig. 18). This could appear when using methods based on detecting change points. In principal, the detector correctly identifies the temporal context of the drift segment, although showing lower scores while the curves drift. Such predictions also score higher values in the sTAUC than the AUC.

Varying length and position of predicted segments

A situation where the sTAUC coincides with the AUC mostly is in when only one true and predicted drift segment exist (see Fig. 19). In cases where the center of the predicted segment coincides with the center of the true segment, the AUC and sTAUC match almost exactly when the length of the predicted segment is varied (see left graphic in Fig. 20). If the predicted segment has fixed length that equals the length of the true segment and the position of its center is varied from 50 to 350, AUC and sTAUC coincide mostly, but the sTAUC shows a faster rise when the predicted segment overlaps with the true segment due to the effects explained in Sect. Lagged prediction.

Appendix D: TAUC for trivial detector

To get a better understanding of the TAUC, we showcase the behavior on trivial detectors based on the structure of the ground truth. Suppose two pair of points $(\text{FPR}_i, \text{OLS}_i)$ and $(\text{FPR}_{i+1}, \text{OLS}_{i+1})$ of the constructed curve. Then the two methods for computing the TAUC are the following (see also Fig. 21):

- **Trapezoidal rule:**
  Construct the curve by linearly interpolating OLS_i and OLS_{i+1} in between FPR_i and FPR_{i+1} and then calculate the area under the curve by using the trapezoidal integration rule.
- **Step rule:**
  Construct the curve by filling the values in between FPR_i and FPR_{i+1} with a constant value of OLS_i and then calculate the area under the curve by using the step rule.

For example, take the trivial detector that always predicts a drift, called `AlwaysGuesser`. Then we receive the two points $(0, 0)$ and $(1, \frac{P}{k})$ as the only two points of the curve, where P denotes the portion of drifts in y and k denotes the number of drift segments in y (see Fig. 22). In case of the step function, the computed score will always be 0, since the constructed curve only contains one step from $[0, 1)$ with a OLS-value of 0, and only reaches a OLS-value of $\frac{P}{k}$ when reaching a FPR of 1 on the x-axis. Hence,the area under this constructed curve is always 0. When using the trapezoidal rule, we linearly interpolate the two obtained trivial points of the curve, thus constructing a line from $(0, 0)$ to $(1, \frac{P}{K})$. The TAUC is then given by the area under this line, which is equal to $\frac{P}{2k}$. Now suppose a detector which never indicates a drift, called `NeverGuesser`. Then we receive $(0, 0)$ as our only point, which does not construct a curve and thus does not have an area under it. Hence, the TAUC for this trivial detection is 0 in both cases.

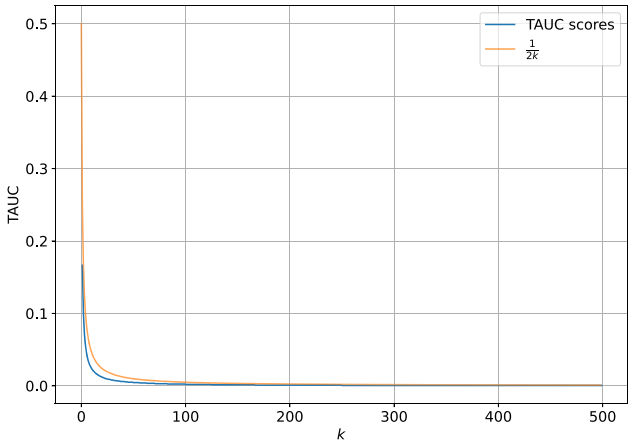


Fig. 23 Visualization of the TAUC with the trapezoidal integration rule, when increasing k, alongside an approximation $\frac{1}{2k}$. The TAUC gets closer to 0 with increasing k

In order to investigate how the TAUC behaves with an increasing number of segments k in y, we simulate such inputs with a trivial detection and compute the resulting values for the TAUC. We choose an input length of $n = 1000$. When using the step rule, the TAUC is always 0 as expected, since the only step always retains its area under the curve of 0. But when looking at the obtained TAUC values when using the trapezoidal integration rule, we can clearly see the TAUC decreasing when k increases.

This decreasing behaviour can be approximated by $\frac{1}{2k}$, since the TAUC for a trivial detection with k segments in case of the trapezoidal rule can be computed with $\frac{P}{2k}$ and $0 < P \leq 1$. Thus, the limit of the TAUC computed with the trapezoidal integration rule with increasing k follows as (see also Fig. 23):

$$\lim_{k \to \infty} \frac{P}{2k} = 0$$

Acknowledgements This work is supported by the Hightech Agenda Bavaria. The authors are grateful to Matthias Burkhardt, Fabian Hueber, Kai Müller, and Ulrich Göhner for helpful discussions. We also thank the anonymous referees for helpful comments and suggestions.

Funding Open Access funding enabled and organized by Projekt DEAL.

Data availability The generated data and implemented algorithms are implemented in a python package `driftbench` which is freely available under https://github.com/edgarWolf/driftbench.

Declarations

Conflict of interest The authors provide no conflict of interest associated with the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Agogino, A., & Goebel, K. (2007). Milling data set, BEST Lab, UC Berkeley, NASA Prognostics Data Repository, NASA Ames Research Center. https://data.nasa.gov/Raw-Data/Milling-Wear/vjv9-9f3x/about_data

Al Assadi, A., Holtz, D., Nägele, F., Nitsche, C., Kraus, W., & Huber, M. F. (2022). Machine learning based screw drive state detection for unfastening screw connections. *Journal of Manufacturing Systems, 65*, 19–32. https://doi.org/10.1016/j.jmsy.2022.07.013

Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., & Steger, C. (2021). The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision, 129*(4), 1038–1059. https://doi.org/10.1007/s11263-020-01400-4

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). JAX: Composable transformations of Python+NumPy programs. http://github.com/jax-ml/jax

Brogan, W. (1991). *Modern control theory*. Prentice-Hall Inc.

Ditzler, G., & Polikar, R. (2011). Hellinger distance based drift detection for nonstationary environments. In *2011 IEEE symposium on computational intelligence in dynamic and uncertain environments* (*CIDUE*) (pp. 41–48). https://doi.org/10.1109/CIDUE.2011.5948491

Duan, Y., Chen, X., Houthooft, R., Schulman, J., & Abbeel, P. (2016). Benchmarking deep reinforcement learning for continuous control. In M. F. Balcan & K. Q. Weinberger (Eds.), *Proceedings of the 33rd international conference on machine learning*. *Proceedings of Machine Learning Research* (*PMLR*), 20–22 Jun 2016 (Vol. 48, pp. 1329–1338). https://proceedings.mlr.press/v48/duan16.html

Fletcher, R. (1987). *Practical methods of optimization* (2nd ed.). Wiley.

Göbler, K., Windisch, T., Pychynski, T., Drton, M., Sonntag, S., & Roth, M. (2024). $causal Assembly$: Generating realistic production data for benchmarking causal discovery. In F. Locatello & V. Didelez (Eds.), *3rd conference on causal learning and reasoning*. *Proceedings of machine learning research* (Vol. 236, pp. 609–642). https://proceedings.mlr.press/v236/gobler24a.html

Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., & Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research, 13*(25), 723–773.

Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology, 143*(1), 29–36. https://doi.org/10.1148/radiology.143.1.7063747

Hesser, D. F., & Markert, B. (2019). Tool wear monitoring of a retrofitted CNC milling machine using artificial neural networks. *Manufacturing Letters, 19*, 1–4. https://doi.org/10.1016/j.mfglet.2018.11.001

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks, 2*(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

Hyvarinen, A., Sasaki, H., & Turner, R. (2019). Nonlinear ICA using auxiliary variables and generalized contrastive learning. In K. Chaudhuri & M. Sugiyama (Eds.), *Proceedings of the twenty-second international conference on artificial intelligence and statistics*. *Proceedings of machine learning research*, *PMLR*, 16–18 Apr 2019 (Vol. 89, pp. 859–868). https://proceedings.mlr.press/v89/hyvarinen19a.html

Khemakhem, I., Kingma, D., Monti, R., & Hyvarinen, A. (2020). Variational autoencoders and nonlinear ICA: A unifying framework. In *Proceedings of the twenty third international conference on artificial intelligence and statistics*, *PMLR* (pp. 2207–2217). https://proceedings.mlr.press/v108/khemakhem20a.html

Kim, Y., Lee, H., & Kim, C. O. (2023). A variational autoencoder for a semiconductor fault detection model robust to process drift due to incomplete maintenance. *Journal of Intelligent Manufacturing, 34*(2), 529–540. https://doi.org/10.1007/s10845-021-01810-2

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational bayes. In *2nd international conference on learning representations*, *ICLR* 2014, Banff, AB, Canada, April 14–16, *conference track proceedings*. https://doi.org/10.48550/arXiv.1312.6114

Lee, J., Lee, Y. C., & Kim, J. T. (2021). Migration from the traditional to the smart factory in the die-casting industry: Novel process data acquisition and fault detection based on artificial neural network. *Journal of Materials Processing Technology, 290*, 116972. https://doi.org/10.1016/j.jmatprotec.2020.116972

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljacic, M., Hou, T. Y., & Tegmark, M. (2025). KAN: Kolmogorov–Arnold networks. In *The thirteenth international conference on learning representations*

Lukats, D., Zielinski, O., Hahn, A., & Stahl, F. (2024). A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics*. https://doi.org/10.1007/s41060-024-00620-y

Mauthe, F., Braun, C., Raible, J., Zeiler, P., & Huber, M. F. (2024). Overview of publicly available degradation data sets for tasks within prognostics and health management. https://doi.org/10.48550/arXiv.2403.13694

Mayr, A., Kißkalt, D., Meiners, M., Lutz, B., Schäfer, F., Seidel, R., Selmaier, A., Fuchs, J., Metzner, M., Blank, A., & Franke, J. (2019). Machine learning in production—potentials, challenges and exemplary applications. In *Procedia CIRP 2019. 7th CIRP Global Web Conference—Towards shifted production value stream patterns through inference of data, models, and technology* (*CIRPe 2019*) (Vol. 86, pp. 49–54). https://doi.org/10.1016/j.procir.2020.01.035

Meiners, M., Kuhn, M., & Franke, J. (2021). Manufacturing process curve monitoring with deep learning. *Manufacturing Letters, 30*, 15–18. https://doi.org/10.1016/j.mfglet.2021.09.006

Meiners, M., Mayr, A., & Franke, J. (2021). Process curve analysis with machine learning on the example of screw fastening and press-in processes. *Procedia CIRP, 97*, 166–171. https://doi.org/10.1016/j.procir.2020.05.220

Nunes, Y., & Guedes, L. A. (2024). Concept drift detection based on typicality and eccentricity. *IEEE Access, 12*, 13795–13808. https://doi.org/10.1109/ACCESS.2024.3355959

Paynabar, K., & Jin, J. J. (2011). Characterization of non-linear profiles variations using mixed-effect models and wavelets. *IIE Transactions, 43*(4), 275–290. https://doi.org/10.1080/0740817X.2010.521807

Raab, C., Heusinger, M., & Schleif, F.-M. (2020). Reactive soft prototype computing for concept drift streams. *Neurocomputing, 416*, 340–351. https://doi.org/10.1016/j.neucom.2019.11.111

Rasmussen, C. E. (2004). *Gaussian processes in machine learning* (pp. 63–71). Berlin: Springer.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning internal representations by error propagation. In *Parallel distributed processing, volume 1: Explorations in the microstructure of cognition: Foundations*. MIT Press

Simard, R., & L'Ecuyer, P. (2011). Computing the two-sided Kolmogorov-Smirnov distribution. *Journal of Statistical Software, 39*(11), 1–18. https://doi.org/10.18637/jss.v039.i11

Tnani, M.-A., Feil, & M., Diepold, K. (2022). Smart data collection system for brownfield CNC milling machines: A new benchmark dataset for data-driven machine monitoring. In *Procedia CIRP 2022. Leading manufacturing systems transformation—Proceedings of the 55th CIRP conference on manufacturing systems 2022* (Vol. 107, pp. 131–136). https://doi.org/10.1016/j.procir.2022.04.022

Wolf, E. (2024). Anomalieerkennung in hoch-dimensionalen Sensordaten, Master's thesis. University of Applied Sciences, Kempten. https://doi.org/10.60785/opus-2338

Yadav, P., Singh, V. K., Joffre, T., Rigo, O., Arvieu, C., Le Guen, E., & Lacoste, E. (2020). Inline drift detection using monitoring systems and machine learning in selective laser melting. *Advanced Engineering Materials, 22*(12), 2000660. https://doi.org/10.1002/adem.202000660

Yong, B. X., Fathy, Y., Brintrup, A. (2020). Bayesian autoencoders for drift detection in industrial environments. In *2020 IEEE international workshop on metrology for industry 4.0 and IoT* (pp. 627–631). https://doi.org/10.1109/MetroInd4.0IoT48571.2020.9138306

Yue, X., Yan, H., Park, J. G., Liang, Z., & Shi, J. (2018). A wavelet-based penalized mixed-effects decomposition for multichannel profile detection of in-line Raman spectroscopy. *IEEE Transactions on Automation Science and Engineering, 15*(3), 1258–1271. https://doi.org/10.1109/TASE.2017.2772218

Zhong, R. Y., Xu, X., Klotz, E., & Newman, S. T. (2017). Intelligent manufacturing in the context of industry 4.0: A review. *Engineering, 3*(5), 616–630. https://doi.org/10.1016/J.ENG.2017.05.015

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.